JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AbstractSelectableChannel

java.lang.Object
  java.nio.channels.spi.AbstractInterruptibleChannel
    java.nio.channels.SelectableChannel
      java.nio.channels.spi.AbstractSelectableChannel

All Implemented Interfaces:

Closeable, AutoCloseable, Channel, InterruptibleChannel

Direct Known Subclasses:

DatagramChannel, Pipe.SinkChannel, Pipe.SourceChannel, SctpChannel, SctpMultiChannel, SctpServerChannel, ServerSocketChannel, SocketChannel

---

```
public abstract class AbstractSelectableChannel
extends SelectableChannel
```

Base implementation class for selectable channels.

This class defines methods that handle the mechanics of channel registration, deregistration, and closing. It maintains the current blocking mode of this channel as well as its current set of selection keys. It performs all of the synchronization required to implement the `SelectableChannel` specification. Implementations of the protected abstract methods defined in this class need not synchronize against other threads that might be engaged in the same operations.

Since:

1.4

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`AbstractSelectableChanne(SelectorProvider provid`	Initializes a new instance of this class.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
final Object	`blockingLock()`	Retrieves the object upon which the

		configureBlocking and register methods synchronize.
final SelectableChannel	**configureBlocking**(boolean block)	Adjusts this channel's blocking mode.
protected final void	**implCloseChannel**()	Closes this channel.
protected abstract void	**implCloseSelectableChann**	Closes this selectable channel.
protected abstract void	**implConfigureBlocking**(boolean block)	Adjusts this channel's blocking mode.
final boolean	**isBlocking**()	Tells whether or not every I/O operation on this channel will block until it completes.
final boolean	**isRegistered**()	Tells whether or not this channel is currently registered with any selectors.
final SelectionKey	**keyFor**(Selector sel)	Retrieves the key representing the channel's registration with the given selector.
final SelectorProvider	**provider**()	Returns the provider that created this channel.
final SelectionKey	**register**(Selector sel, int ops, Object att)	Registers this channel with the given selector, returning a selection key.

Methods declared in class SelectableChannel

register, validOps

Methods declared in class AbstractInterruptibleChannel

`begin`, `close`, `end`, `isOpen`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

AbstractSelectableChannel

```
protected AbstractSelectableChannel(SelectorProvider provider)
```

Initializes a new instance of this class.

Parameters:

`provider` - The provider that created this channel

Method Details

provider

```
public final SelectorProvider provider()
```

Returns the provider that created this channel.

Specified by:

`provider` in class `SelectableChannel`

Returns:

The provider that created this channel

isRegistered

```
public final boolean isRegistered()
```

Description copied from class: `SelectableChannel`

Tells whether or not this channel is currently registered with any selectors. A newly-created channel is not registered.

Due to the inherent delay between key cancellation and channel deregistration, a channel may remain registered for some time after all of its keys have been cancelled. A channel may also remain registered for some time after it is closed.

Specified by:

isRegistered in class SelectableChannel

Returns:

true if, and only if, this channel is registered

keyFor

```
public final SelectionKey keyFor(Selector sel)
```

Description copied from class: SelectableChannel

Retrieves the key representing the channel's registration with the given selector.

Specified by:

keyFor in class SelectableChannel

Parameters:

sel - The selector

Returns:

The key returned when this channel was last registered with the given selector, or null if this channel is not currently registered with that selector

register

```
public final SelectionKey register(Selector sel,
                                   int ops,
                                   Object att)
                            throws ClosedChannelException
```

Registers this channel with the given selector, returning a selection key.

This method first verifies that this channel is open and that the given initial interest set is valid.

If this channel is already registered with the given selector then the selection key representing that registration is returned after setting its interest set to the given value.

Otherwise this channel has not yet been registered with the given selector, so the register method of the selector is invoked while holding the appropriate locks. The resulting key is added to this channel's key set before being returned.

Specified by:

register in class SelectableChannel

Parameters:

sel - The selector with which this channel is to be registered

`ops` - The interest set for the resulting key

`att` - The attachment for the resulting key; may be `null`

Returns:

A key representing the registration of this channel with the given selector

Throws:

`ClosedSelectorException` - If the selector is closed

`IllegalBlockingModeException` - If this channel is in blocking mode

`IllegalSelectorException` - If this channel was not created by the same provider as the given selector

`CancelledKeyException` - If this channel is currently registered with the given selector but the corresponding key has already been cancelled

`IllegalArgumentException` - If a bit in the `ops` set does not correspond to an operation that is supported by this channel, that is, if `set & ~validOps() != 0`

`ClosedChannelException` - If this channel is closed

implCloseChannel

```
protected final void implCloseChannel()
                                throws IOException
```

Closes this channel.

This method, which is specified in the `AbstractInterruptibleChannel` class and is invoked by the `close` method, in turn invokes the `implCloseSelectableChannel` method in order to perform the actual work of closing this channel. It then cancels all of this channel's keys.

Specified by:

`implCloseChannel` in class `AbstractInterruptibleChannel`

Throws:

`IOException` - If an I/O error occurs while closing the channel

implCloseSelectableChannel

```
protected abstract void implCloseSelectableChannel()
                                            throws IOException
```

Closes this selectable channel.

This method is invoked by the `close` method in order to perform the actual work of closing the channel. This method is only invoked if the channel has not yet been closed, and it is never invoked more than once.

An implementation of this method must arrange for any other thread that is blocked in an I/O operation upon this channel to return immediately, either by throwing an exception or by returning normally.

Throws:

`IOException` - If an I/O error occurs

isBlocking

```
public final boolean isBlocking()
```

Description copied from class: `SelectableChannel`

Tells whether or not every I/O operation on this channel will block until it completes. A newly-created channel is always in blocking mode.

If this channel is closed then the value returned by this method is not specified.

Specified by:

`isBlocking` in class `SelectableChannel`

Returns:

`true` if, and only if, this channel is in blocking mode

blockingLock

```
public final Object blockingLock()
```

Description copied from class: `SelectableChannel`

Retrieves the object upon which the `configureBlocking` and `register` methods synchronize. This is often useful in the implementation of adaptors that require a specific blocking mode to be maintained for a short period of time.

Specified by:

`blockingLock` in class `SelectableChannel`

Returns:

The blocking-mode lock object

configureBlocking

```
public final SelectableChannel configureBlocking(boolean block)
                                          throws IOException
```

Adjusts this channel's blocking mode.

If the given blocking mode is different from the current blocking mode then this method invokes the `implConfigureBlocking` method, while holding the appropriate locks, in order to change the mode.

Specified by:

`configureBlocking` in class `SelectableChannel`

Parameters:

`block` - If `true` then this channel will be placed in blocking mode; if `false` then it will be placed non-blocking mode

Returns:

This selectable channel

Throws:

`ClosedChannelException` - If this channel is closed

`IOException` - If an I/O error occurs

implConfigureBlocking

```
protected abstract void implConfigureBlocking(boolean block)
                                       throws IOException
```

Adjusts this channel's blocking mode.

This method is invoked by the `configureBlocking` method in order to perform the actual work of changing the blocking mode. This method is only invoked if the new mode is different from the current mode.

Parameters:

`block` - If `true` then this channel will be placed in blocking mode; if `false` then it will be placed non-blocking mode

Throws:

`IOException` - If an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.